Dynamic Prototype Network Based on Sample Adaptation for Few-Shot Malware Detection

Yuhan Chai, Lei Du, Jing Qiu, Lihua Yin, and Zhihong Tian, *Senior Member, IEEE*

Abstract—The continuous increase and spread of malware have caused immeasurable losses to social enterprises and even the country, especially unknown malware. Most existing methods use predefined class samples to train models, which cannot handle unknown malware detection. In this paper, we formalize unknown malware detection as a Few-Shot Learning problem. However, the existing model cannot dynamically adjust the model parameters according to the samples and does not deeply consider the influence of the correlation between samples, so it achieves sub-optimal performance. We propose a Dynamic Prototype Network based on Sample Adaptation for few-shot malware detection (DPNSA). Specifically, we use dynamic convolution to realize dynamic feature extraction based on sample adaptation. Secondly, we define the class feature (prototype) as the mean of the dynamic embedding of all malware samples of each class in the support set. Then, a dual-sample dynamic activation function is proposed, which uses the correlation of the dual-sample to reduce the impact of unrelated features between samples on the metric. Finally, we use the metric-based method to calculate the distance between the query sample and the prototype to realize malware detection. Experiments show that our method outperforms the existing few-shot malware detection models and achieves significant improvement.

Index Terms—Feature representation, neural nets, similarity measures, security

1 Introduction

With the increase of cyberattacks, the resulting sensitive data leakage incidents continue to hit the headlines on media. This causes heavy economic losses to users, enterprises and even the country. The attacker can formulate new attack methods based on the defender's defense measures, whereas the defender can effectively defend against new attack methods based on attack strategy. Amid the game between the attacker and the defender, unknown new attack threats are constantly being produced. The AV-TEST report pointed out that the number of newly-developed malware has increased by nearly 110 million in 2020. This is the second highest number since the record in 1984, which shows that we still have a long way to go on the road to ensure information security. It is urgent and necessary to strengthen the research of unknown malware detection technology [1].

In recent decades, malware detection methods have been extensively studied. These methods can be split into three clusters: static methods [1], [4], [56], dynamic methods [5], [6], [7], and visualization methods [8], [9]. The static method usually extracts malware features through static analysis techniques; these extracted features are analyzed or compared with feature libraries to determine whether there is malicious behavior. Some malware, however, can circumvent static detection by packaging and obfuscation. To solve this issue, dynamic method is proposed, to dynamically analyze the malware in an isolated sandbox environment and judge whether it is malware by the runtime behavior. Compared with the static method, the dynamic detection method is more time-consuming. Some, intending to detect malware based on image texture features, proposed the visualization methods. In this method, a malware binary file is converted into an image representation, and the texture feature of the image is used for malware detection [21]. Compared with static and dynamic methods, the visualization method requires neither disassembly nor dynamic execution. In this paper, we employes the visualization-based method for unknown malware detection.

Due to explosive growth of malware, traditional malware detection methods can no longer fulfil the necessities for efficient malware detection in big-data scenarios. Thanks to the exploitation of deep learning and its successful application to natural language processing [10], [11], [12] and computer vision [13], [14], deep learning has been increasingly applied to malware detection, and have achieved well-pleasing results [3], [4], [5], [6], [7], [8], [9]. However, these methods suffer two major shortcomings: the first is that a model with good detection performance can be trained only when a great many of class-labeled data are available; the second is the inability to cope with the increasing number of unknown malware. The cost of labeling a large amount

- *Yuhan Chai, Jing Qiu, Lihua Yin, and Zhihong Tian are with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, Guangdong 510006, China. E-mail: chaiyuhan@e.gzhu.edu.cn, {qiujing, yinlh, tianzhihong}@gzhu.edu.cn.*
- *Lei Du is with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, Guangdong 518055, China, and also with the Peng Cheng Laboratory, Shenzhen, Guangdong 518066, China. E-mail: leidu_close@163.com.*

Manuscript received 23 September 2021; revised 26 December 2021; accepted 7 January 2022. Date of publication 13 January 2022; date of current version 3 April 2023.
This work was supported in part by Guangdong Province Key Research and Development Plan under Grant 2019B010136003, in part by National Key research and Development Plan under Grant 2018YFB1800702, in part by the National Natural Science Foundation of China under Grants U20B2046, U1636215, 61871140, and U1803263, in part by Guangdong Higher Education Innovation Group under Grant 2020KCXTD007 and in part by Guangzhou Higher Education Innovation Group under Grant 202032854, Guangdong Province Universities and Colleges Pearl River Scholar Funded Scheme (2019). (Corresponding author: Jing Qiu.)
Recommended for acceptance by Y. Zhang.
Digital Object Identifier no. 10.1109/TKDE.2022.3142820

of data is undoubtedly huge, and the accuracy of labeling is also a big challenge. Moreover, in real application scenarios, unknown malware will keep emerging. Most existing malware detection models cannot handle unknown malware detection well due to the limited number of predefined classes. One conventional solution is to collect a large number of samples of the new class and use the original data set for retraining. However, this method is costly, and the collection and labeling of unknown malware is also an extremely arduous task. Thus, it is necessary and urgent to research unknown malware detection methods [2].

Intuitively, humans can use previous experience to grasp new knowledge by learning a few examples. Few-Shot Learning (FSL) means learning from limited examples. As unknown malware is difficult to obtain, unknown malware detection is considered a few-shot problem naturally. Therefore, we provide a different perspective to formalize the malware detection problem as a FSL problem. In the FSL scenario, an N-way K-shot task is composed of N classes and K training examples per class. Another testing set sampled from the same N classes is provided to evaluate the classifier.

Few-shot learning can train a reliable and effective model based on a handful of labeled data of unknown malware sample in the absence of sufficient labeled data, and realize unknown malware detection. However, this is not a simple matter because of the following two key issues. The first is the need to consider how to take full advantage of a handful of samples to prevent the model from over-fitting and maximize the model's characterization capacity. Shortage of data will lead to serious overfitting problems when the parameters of the model are excessively large. The lightweight neural networks provide a solution to the overfitting problem, but the model performance and expression capacity will be compromised due to its low computational ability. The second key issue is the need to fully consider the correlation between samples and ignore interference information that is not conducive to malware detection. As malware of the same class shares similar function calls, the same class of malware has certain similarities. Also, malware in the same family shares similar coding ways, malware in the same family bear a certain resemblance. For example, for samples that belong to the same family but are not in the same class, this class of malware mixes similar features with dissimilar features. There are differences in similarities, and similarities in differences, which can be considered interference information that is not conducive to malware detection.

To solve the above-mentioned questions, we propose a Dynamic Prototype Network based on Sample Adaptation for few-shot malware detection (DPNSA). Firstly, a dynamic convolutional neural network is introduced in a few-shot learning scenario for the first time. It realizes dynamic feature embedding based on sample adaptation and perfectly expresses the semantic information in the sample. It enables the improvement of the characterization ability of the model while minimizing the growth of calculation workload. Then, we propose a dual-sample dynamic activation function to resolve the issue that the inter-sample interference information is not conducive to effective metrics. The dual-sample dynamic activation function uses the correlation between the malware class features and query sample features to activate them dynamically. It can ignore the interference information between samples and reduce the influence of irrelevant features between samples on the metric. Finally, the metric-based method is applied to compute the distance between the query sample and the malware class prototype to realize effective malware detection.

A massive number of experiments were performed on a few-shot malware detection datasets to prove the effectiveness of our approach. Our main contributions are summarized as follows:

- We propose a dynamic prototype network based on sample adaptation for few-shot malware detection, which solves the problem of unknown malware detection. The meta-knowledge formed by the existing task experience is used to quickly learn a handful of samples of unknown malware, so as to achieve effective unknown malware detection.
- We investigated the dynamic convolution into few-shot learning scenarios. This is used to solve the over-fitting problem caused by the handful of samples and the insufficient expression ability of the lightweight neural network. To the best of our knowledge, this is the first time that a dynamic convolutional neural network has been investigated in few-shot learning scenario.
- We propose a dual-sample dynamic activation function to resolve the issue that the interference information between samples is not conducive to the effective metric. At the same time, it can also effectively improve the expression capacity of the model without significantly increasing the amount of calculation.
- The experimental results on the few-shot malware dataset demonstrate that our approach is dramatically better than the benchmark model on 1-shot, 5-shot, and 10-shot scenarios, which demonstrates the effectiveness of DPNSA and achieves significant improvement.

The rest of this paper is arranged as follows. Section II reviews the related work on malware detection, few-shot learning and few-shot malware detection. Section III formally emphasizes the details of the DPNSA model. In Section IV, the performance of the DPNSA model, including the design of parameters and the discussion of experimental results, is evaluated. In Section V, conclusions of the present work and future research directions are provided.

2 Related Work

This section briefly reviews related work on malware detection, few-shot learning and few-shot malware detection.

2.1 Malware Detection

For the past few years, deep learning has seen increasing adoption and shown strength in various tasks. It has also been increasingly applied to malware detection. Existing deep learning-based malware detection methods include static malware detection, dynamic malware detection, visualization-based malware detection, and combined static-dynamic malware detection.

In terms of static malware detection, Gibert *et al.* [3] proposed a hierarchical convolutional network (HCN), which uses the malware assembly language source code in the executable file to extract mnemonic-level and function-level features for malware detection. The graph structure can better show the execution logic of the program. Yan *et al.* [4] proposed MAGIC for malware detection: first, a control flow graph was built according to the execution order of the disassembled code; then, the attributes for each node in the control flow graph were defined to convert the code features into numerical values; finally, the deep graph CNN was used for malware detection. Zhu *et al.* [56] proposed to how to learn effective feature representation from the original data is solved by combining the Merged Sparse Auto-Encoder and the Stacked Denoising Auto-encoders. These two methods belong to unsupervised learning, they solve the problem that feature learning depends on prior knowledge or experts.

There are also many dynamic malware detection methods. Pascanu *et al.* [15] proposed to extract semantic information which contained the API call sequence by using the recurrent neural network (RNN) and echo state network (ESN) to achieve results of malware. Kolosnjaji *et al.* [16] applied the combination of CNN and RNN to extract semantic information in the system call sequence, which improves the performance of malware classification. Tobiyama *et al.* [5] proposed to apply RNN to extract semantic information from the API call sequence and API return value; then use the converted image output from the model to classify malware. Chai *et al.* [17] proposed a malware detection framework based on local and global features: first, the local semantic information in the API call sequence was extracted by a stacked CNN; then, a statistical-based method was used to construct the API call sequence into an API semantic graph, and the graph convolutional network is devoted to extract the global semantic information contained in the API semantic graph. The combination of these two features is devoted to malware detection. Lu *et al.* [6] proposed to use the bidirectional residual LSTM model to extract the semantic information from the API sequence data and the random forest model to extract the semantic information from the API statistical features to determine the malware classification results. Xiao *et al.* [7] proposed the BDLF framework, which uses the SAEs model to extract high-level semantic features from the behavior graph, and then feeds it into the classifier to detect malware in the Internet of Things system. Zhang *et al.* [18] proposed an effective deep neural network (DNN) architecture for malware detection, and the cost of feature extraction method is inexpensive: first, the Cuckoo sandbox is used to run the collected PE files to obtain sample behavior information; Secondly, the hash method is used to encode the API name, category, and parameters separately, and stitch all the features; then, the high-dimensional hash feature of each API call is convert into low-dimensional space through multiple gated-CNNs, and select important and relevant information; finally, a bi-LSTM model is used to capture the sequential correlation of API calls in the sequence, and hence realize malware detection.

There is a potential relationship between the static and dynamic API call sequence of the malware. This relationship is different in syntax, but similar in semantics. Therefore, static-dynamic analysis combined malware detection techniques have been proposed. Yen *et al.* [7] combined static and dynamic features with a DNN for malware classification on Windows. Gibert *et al.* [19] proposed a multi-modal deep learning framework named HYDRA for malware detection. The framework achieves malware detection and classification by fusing three different features: first, the hexadecimal byte sequence is extracted from the original data of the malware, the assembly language instruction sequence is extracted from the static data of the malware, and the API function call sequence is extracted from the dynamic data of the malware; then, the features extracted from the data modal are fused into a shared representation; finally, the end-to-end training of the model is carried out to realize the malware detection. Snow *et al.* [23] proposes a multi-model deep learning framework in end-to-end manner for malware classification. In their framework, three diverse deep neural network structures — DenseNetwork, RNN and CNN are used jointly to learn significative features from various attributes of malware.

Many visualization-based malware detection methods based on deep learning have also been proposed. Nataraj *et al.* [21] for the first time converted the executable file of the malware into a corresponding gray-scale image to generate feature vectors based on image texture features, which provides a potential solution to malware detection. This method creates a precedent for visualization-based malware detection without disassembly or code execution. He *et al.* [24] convert malware files into image representations that are devoted to malware detection. CNN including the Spatial Pyramid Pooling Layer (SPP) solves the issue of diverse input image sizes. Yuan *et al.* [22] proposed a byte-level malware classification method based on Markov images—MDMC: first, the malware binary file is converted into a Markov image according to the byte transition probability matrix; then, a deep convolutional neural network (DCNN) is devoted to achieving malware detection according to the Markov image. Bhodia *et al.* [9] combined image analysis and transfer learning, using models pre-trained on large image datasets for malware detection. Kim *et al.* [8] proposed the Transfer Deep Convolutional Generative Adversarial Network (tDCGAN), which generates fake malware to allow the model to learn to distinguish real malware. In their method, a deep autoencoder served as a generator to generate fake malware, and the trained discriminator obtains malware detection capabilities.

Although the existing deep learning-based malware detection methods show strong capabilities, they rely heavily on large-scale labeled data. These methods cannot cope with the increasing number of unknown malware detection. Here, we focus on malware detection in few-shot scenarios.

2.2 Few-Shot Learning

Meta-learning [25] has been widely served as solve few-shot learning problems. FSL is to learn the tasks constituted by the existing datasets, and makes full use of the learned knowledge (that is, meta-knowledge) to quickly recognize unknown classes for new FSL tasks. The purpose of FSL is to use a handful of labeled samples of unknown classes to identify unlabeled samples.

The existing FSL methods based on meta-learning can be roughly split into two clusters: optimization-based methods and metric-based methods.Optimization-based methods aim to search for a set of model parameters through several gradient descent steps to be suitable for various tasks. Optimization-based approaches, including MAML [26], which is applicable to any model that can be updated using gradient descent. This method uses the already trained parameters and only needs to fine-tune the gradient update a few steps to achieve good results on the new task. On the basis of MAML, a simpler meta-learning algorithm called Reptile for parameter initialization is proposed [30]. Meta-SGD [27] is an improvement of MAML, learning the weight initialization and the update step size of the learner. SNIAL [28] is integrated by an interleaved temporal convolution layer and a causal attention layer that achieves SOTA capability. The task-agnostic meta-learning (TAML) paradigm [29] is an entropy-based method that meta-learns the unbiased initial model with the maximum nondeterminacy of the output label to prevent it from excessive performance in classification tasks.

The metric-based method is to map the input sample into a new embedding space by learning an embedding function, and the similarity metric function is used to distinguish different classes of samples. This embedding function is the meta-knowledge learned in the task. When a new class sample arrives, we only need to map the required sample through the embedding function so that the similarity metric is used for comparison in the embedding space, thereby realizing unknown class classification. Popular metric-based methods include ProtoNet [31], RelationNet [32], and MatchingNet [33]. To better learn a good embedding and an appropriate metric method, Oreshkin *et al.* [34] proposed TADAM which introduces the feature extraction according to different tasks, and learns the metric space of task-related in the end-to-end optimization manner of auxiliary task co-training. Tseng *et al.* [36] proposed a method that uses feature-based transformation layers to enhance image features through affine transformation in the training phase to simulate various feature distributions in different domains, and solves the problem that the metric-based method cannot show good generalization in across-domain applications. Ye *et al.* [37] proposed a transformer based on the self-attention mechanism. By constructing the correlation between different tasks within the set, the task-independent feature information is converted into task-related feature information, thereby improving the recognition accuracy on the target task. Due to the scarcity of data in training tasks, it is not easy to own a good metric space. To overcome this problem, Wang *et al.* [38] combined softmax loss and triplet loss to learn a distinguishable metric space and improve the performance of the metric learning method. Gidaris *et al.* [39] combined additional self-supervised learning tasks with few-shot classification tasks. The two tasks share a feature extraction network, and the self-supervised learning task is used to improve the characterization ability of the feature extraction network, thereby improving the effect of few-shot classification tasks. Hou *et al.* [35] proposed a cross-attention network for few-shot classification, in which the cross-attention module solved the problem of the attention misalignment between the support sample and the query sample. Zhang *et al.* [40] transformed the one-shot classification problem into an optimal matching problem, and proposed a few-shot classification method that uses EMD as a measure of correlation between images.

The FSL method based on meta-learning has been proven to perform well in various tasks, and metric-based methods have achieved optimal capability in few-shot learning. However, few-shot malware detection has rarely been studied. Moreover, the cost of obtaining massive labeled data is very high in the network security, and the network security problem in the few-shot scenario needs to be solved urgently.

2.3 Few-Shot Malware Detection

To solve the problem that unknown malware samples are scarce and difficult to detect, Tang *et al.* [41] proposed the ConvProtoNet model to classify malware. The model had three modules: an embedding module is used to embed samples into the feature space, a convolution induction module for calculating malware class prototypes, and a SoftMax classifier generator that uses revised cosine similarity functions to calculate the distance from samples to prototypes. To solve the problem that many malware families have only a small amount of malware, Bai *et al.* [42] introduced a method based on the siamese network. The method trains an effective multi-layer perceptron (MLP) network, and then embeds malware into a real-valued, continuous vector space to judge whether it is malware of the same or different families. Trung *et al.* [43] proposed a method to classify unknown malware using a memory-augmented neural network: first, the N-gram method is used to extract the characteristics of the malware; then the word2vec model is used to vectorize the features; and then only a single or a few samples of unknown malware are used to achieve malware classification through the memory-augmented neural network with the LRUA model. Deng *et al.* [44] proposed an attention-based transduction learning network that can learn label information from a handful of samples in each malware family. The network consists of three parts: firstly, the malware is converted into resizable grayscale images; Secondly, a CNN is used for feature embedding; then a Gaussian similarity graph based on the attention mechanism is constructed. The tag information flows from the support set to the query set, thereby realizing malware classification. Zhu *et al.* [45] proposed using the siamese neural network to achieve malware variant detection with only a handful of samples. Hsiao *et al.* [46] introduced the siamese neural network for one-shot malware image classification tasks: first, the malware is converted into resizable grayscale images; then, the siamese neural network is used to calculate the similarity between samples; and at last, the malware is classified by sorting the similarities.

In summary, there are just a few methods for few-shot malware detection, and most of them directly use the classical prototype network, matching network, memory-augmented neural networks, or the siamese network. These methods fail to take into full consideration the self-adaptation of samples and the correlation between samples, resulting in sub-optimal detection accuracy.

Our work is a metric learning-based method. Unlike the existing methods based on metric learning to extract support and query sample features statically, our method uses

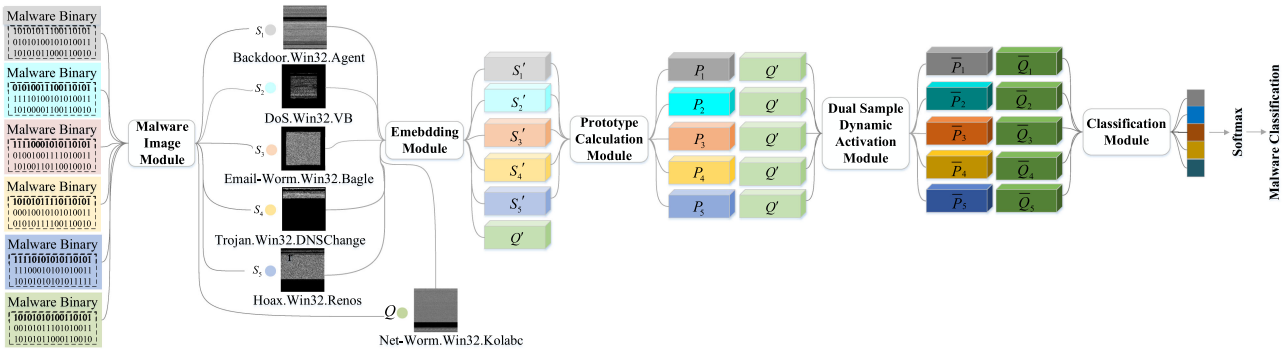


Fig. 1. The framework of the proposed DPNSA for few-shot malware detection.

a dynamic convolution in few-shot scenarios to realize dynamic feature embedding based on sample adaptation for the first time. In addition, we consider the correlation between class features and query sample features. In this paper, we propose a dual-sample dynamic activation function. It uses the correlation between samples to dynamically activate the class features and query sample features to reduce the impact of irrelevant inter-sample features on the metric.

3 Methodology

We Firstly present the problem definition for malware detection in a few-shot scenario in this section. Then, the overall framework of our dynamic prototype network based on sample adaptation is described in detail.

3.1 Problem Definition

Few-shot learning usually has a training set, a validation set, and a test set. The label spaces between the three datasets are disjoint. The training set comprises multiple groups of support sets and query sets, and the support set and query set partake in the same label space. The validation and test sets are similarly composed of support sets and query sets. Few-shot learning is to classify unlabeled query samples based on a training set with many labeled samples and a support set with a handful of labeled samples; support set and query set are in the validation(test) set. Specifically, the training set is first used for meta-learning to extract transferable knowledge; then, the meta-knowledge gained from prior tasks is used to quickly learn a handful of samples of unknown new classes the support set of in the test set to classify the query set.

Few-shot learning is the application of meta-learning to supervised learning. Meta-learning involves two stages: the meta-training stage and the meta-testing stage. In the meta-training stage, the training set is decomposed into different meta-tasks to learn the generalization ability of the model when the class changes. In the meta testing stage, the classification can be completed without changing the existing model when faced with a brand new class. Following the latest few-shot learning setting, if the support set is composed of N classes and K labeled samples for each class, then the few-shot learning problem is called N-way K-shot. That is, the model needs to learn how to distinguish these N classes based on $N*K$ data.

We adopt the episodic training strategy proposed by Vinyals *et al*. [33]. The episodic strategy used in the meta-training phase needs to simulate the settings in the meta-testing phase. We define meta-task function $\mathbf{MT}: (\mathbf{S}, \mathbf{Q}) \rightarrow \mathbf{Y}$. In the meta-training phase, N classes will be randomly selected from the training set, and K samples for each class will be used as the support set $S = \{S_1, S_2, \ldots, S_N\}$, including $S_1 = (s_1^1, s_1^2, \ldots, s_1^K), \ldots, S_N = (s_N^1, s_N^2, \ldots, s_N^K)$. The labels corresponding to the N classes are $Y = \{y_1, y_2, \ldots, y_N\}$. Then, from the N classes, W samples that do not exceed the number of the remaining samples are selected as the query set $Q = \{Q_1, Q_2, \ldots, Q_N\}$, including $Q_1 = (q_1^1, q_1^2, \ldots, q_1^W), \ldots, Q_N = (q_N^1, q_N^2, \ldots, q_N^W)$. Finally, a meta-task function $\mathbf{MT}$ is constructed for model training, and then the trained model is used to predict the sample labels in the query set. We define S_n as the support set of the n^{th} class, and q_n^w as a sample of the query set of the n^{th} class, where $1 \leq n \leq N$ and $1 \leq w \leq W$. The key is to better denote each malware class of support set S_n and query set sample q_n^w, and calculate the similarity between them for metric-based few-shot malware detection.

3.2 Overview of Designed DPNSA

In this section, we explain our proposed Dynamic Prototype Network based on Sample Adaptation (DPNSA) at great length. As shown in Fig. 1, 5-way 1-shot as an example, our model is composed of five modules: malware image module, feature embedding module, prototype calculation module, dual-sample dynamic activation module and metric learning module. Malware image module: Given a malware executable file, its byte sequence is converted into a fixed-size grayscale image as a malware image. Feature embedding module: Given a malware image, a neural network is used to encode the image and texture features into dynamic semantic feature embedding. Prototype calculation module: Based on the prototype network, we calculate the mean of the dynamic embedding of all malware samples of each class in the support set as the class feature or prototype of each class. Dual-sample dynamic activation module: The correlation between the malware class feature and the query sample feature is calculated to learn a new dual-sample dynamic activation function, which is used for nonlinear dynamic activation of the two samples. Then a convolutional layer and a max-pooling layer are used to extract important information. Metric learning module: The distance between the updated malware class feature (prototype) and the query sample feature is calculated to realize the unknown malware detection.

Malware Image Module

The advantage of using malware images is reduce the need for domain expert knowledge. We took in the idea proposed

by Nataraj *et al.* [21] to convert malware executable files into grayscale images that are easier to process, and the texture features in the grayscale image are then used for malware detection. To convert binary malware into an image, we first convert the byte sequences of the binary malware codes into one-dimensional raw pixel data streams, which are then converted into two-dimensional grayscale images. We use the same method as in [41] to convert the malware executable file into a fixed-width and fixed-length square image.

Dynamic Feature Embedding Module

In the few-shot learning scenario, the sample size is small, and data are scarce, which is likely to lead the model to overfitting when the model has a great many of parameters. The lightweight neural network is a viable solution to the overfitting problem caused by a handful of labeled data. Still, the performance and expression capacity will be impaired because of the low calculation ability of the network. Furthermore, in static convolution, the same convolution kernel is used to perform the same convolution operation on all input malware images, but in fact, the malware images are different from each other and have their own characteristics.

Therefore, to solve the problem of overfitting without sacrificing the network's expression capacity, we introduce a lightweight network with a dynamic convolution layer [47] in a few-shot learning scene to realize the dynamic feature embedding of samples based on sample adaption. It can prevent the model from overfitting and improve its expression capacity while minimizing the increase of model parameters. In dynamic convolution, the squeeze-and-excitation [48] is employed to calculate the convolution kernel attention based on the input of different malware images, and the convolution parameters are then adaptively adjusted so that more suitable convolution parameters can be selected to perform different convolution operations on them for different malware images. Since the convolution kernel is a function of the input, the dynamic convolution is no longer a linear function. By superimposing the convolution kernel in a non-linear manner by attention, it can increase the expression capacity of the model without increasing the depth or width of the network.

Specifically, given a malware image, we use a lightweight dynamic convolutional neural network containing static convolution and dynamic convolution to embed the original malware image features into a low-dimensional continuous space to capture the dynamic semantic information of the malware. The dynamic feature of each malware sample in each class in the support set is embedded as $s_n^{k^{'}} = E(s_n^k)$. The sum of dynamic features embedding of all samples of each class is as $S_n^{'} = \sum s_n^{k^{'}}$. The dynamic feature of each malware sample in each class in the query set is embedded as $Q^{'} = E(q_n^w)$, where $1 \leq n \leq N, 1 \leq k \leq K, 1 \leq w \leq W$, $E(\cdot)$ is the dynamic embedding function and $K = 1$ as shown in Fig. 1. The architecture of the dynamic feature embedding module is shown in Fig. 2.

Prototype Calculation Module

We use the dynamic feature embedding of malware images to compute prototypes for each class in the support set. The central idea of the prototype network is to average the embedding of all samples of each class as the embedding of class, also called a prototype, to represent each malware class. The general method of calculating the prototype is $P_n = \frac{S_n^{'}}{|S_n|}$, which is to average the dynamic feature embedding of all malware images of each class in the support set, where P_n is the prototype of n^{th} class, $|S_n|$ is the sample number of n^{th} class, and $S_n^{'}$ is the sum of dynamic features embedding of all samples of n^{th} class.

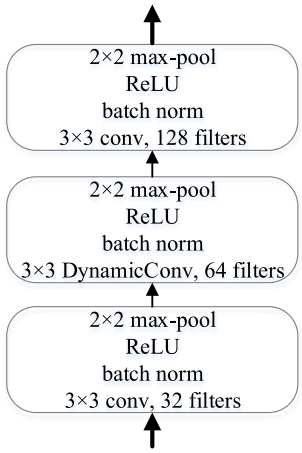


Fig. 2. The architecture of the dynamic feature embedding module.

Dual-Sample Dynamic Activation Module

The architecture of the dual-sample dynamic activation module is shown in Fig. 3. First, the dual-sample activation function is used to dynamically activate the two samples in a nonlinear manner according to the correlation between the two samples. It can reduce the impact caused by irrelevant features between samples on the metric, and can also improve the model's expression capacity. Then, the convolutional layer and the max-pooling layer are used to extract important feature information of the sample. The motivation, design and implementation details of the dual-sample dynamic activation function will be explained below.

Motivation for proposing dual-sample dynamic activation function

One challenge in classification is to map the same class of sample more closely, and to map different classes of samples distantly. The metric can be employed to better identify whether two samples belong to the same class or different classes. Specifically, the malware classification task is different from regular classification tasks. Malware class labels indicate not only the family, but also the class of the malware. Since the same class of malware shares similar function calls, the same class of malware shares certain similarities. Besides, the similarities in coding ways of malware in the same family, malware in the same family have certain similarities.

There are mainly four situations according to whether the family and class are the same as follows. First, for samples that belong to both the same family and the same class, the malware of the same family has similar characteristics, and the samples of the same class contain similar code fragments, so the correlation between the two samples for this type of situation is stronger. It is necessary to consider how to learn the compact features of this class of malware. Second, for samples that are neither in the same family nor in the same class, the correlation between the two samples is the weakest, and it is necessary to consider how to learn the deeply separable characteristics of this class of malware.

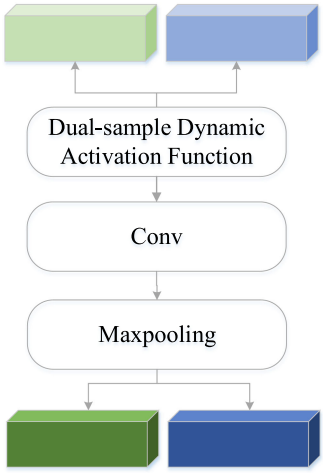


Fig. 3. The architecture of dual-sample dynamic activation module.

Third, for samples that belong to the same family but are not in the same class, they are in fact different classes of malware samples. This class of malware mixes similar features with dissimilar features, and it is difficult to well separate the different classes of malware samples. Fourth, for samples that are not in the same family but belong to the same class, although they are different families, they are malware samples of the same class. This class of malware also mixes similar features with dissimilar features. There are differences in similarities, and similarities in differences, which can be considered as interference information that is not conducive to the measurement between samples.

The traditional activation function in the deep learning method is to pass the same nonlinear transformation to different malware samples, which has a negative impact on the model's capacity to ignore the interference information and learn distinguishing features. Therefore, it is challenging to map these samples more separately through the same nonlinear transformation function for samples of different classes in different families, samples of different classes in the same family, and samples of the same class in different families. It is difficult to map samples belonging to the same family and class more compactly. In order to ignore the interference of irrelevant features and improve the distinguishability of the learned features, we propose to develop a new activation function to dynamically and nonlinearly transform two samples based on the correlation of the two samples.

Design of Dual-Sample Dynamic Activation Function

Inspired by Chen *et al*. [49], we propose a new activation function, the dual-sample dynamic activation function. It further extends from the single-sample static activation function and dynamic activation function to the dual-sample dynamic activation function. The relevance between samples is improved by improving the semantic characteristics of the class shared by the samples. According to the correlation between two samples, an appropriate dynamic activation function is adaptively learned for the two samples. Then, the two samples are dynamically activated separately. Specifically, the dual-sample dynamic activation function can effectively ignore the interference information between samples, reduce the impact of irrelevant features on the metric, and improve the distinguishability of learned features, thereby improving detection accuracy. The slope and variables of the dual-sample dynamic activation function depend on the dynamic changes of the two input samples. Therefore, the dual-sample dynamic activation function has a more flexible nonlinear transformation than the traditional rectified linear unit (ReLU) activation function.

Definition: Given two input feature vectors $Q^{'}$ and P_n, $P_n = \{P_{n,c}\}(n = 1, \ldots, N; c = 1, \ldots, C)$ defines each class prototype feature vector (or tensor), with C channels. $Q^{'} = \{Q^{'}_c\}(c = 1, \ldots, C)$ defines the feature vector (or tensor) of each query sample and has C channels. The definition of the dual-sample dynamic activation function $D_{\alpha(Q^{'},P_n)}(Q^{'}_c, P_{n,c})$ with learnable parameters $\alpha(Q^{'}, P_n)$ proposed in this paper is shown in the following formula. The dual-sample dynamic activation function is related to the two input feature vectors $Q^{'}$ and P_n.

$$\overline{Q}_c, \overline{P}_{n,c} = D_{\alpha(Q^{'},P_n)}(Q^{'}_c, P_{n,c}) = \underset{1 \leqslant f \leqslant F}{max} \left\{ a^f_c(Q^{'}, P_n)(Q^{'}_c, P_{n,c}) + b^f_c(Q^{'}, P_n) \right\} \quad (1)$$

where the hyper function $\alpha(Q^{'}, P_n)$ is used to calculate the activation parameters of the dynamic activation function between the two samples, namely the slope a^f_c and the constant b^f_c. These two activation parameters are not static, but change constantly with the two input feature vectors $Q^{'}$ and P_n. The dual-sample dynamic activation function $D_{\alpha(Q^{'},P_n)}(Q^{'}_c, P_{n,c})$ uses a hyper function $\alpha(Q^{'}, P_n)$ to generate activation for all channels of the correlation of the two input feature vectors.

$$\alpha(Q^{'}, P_n) = [(a^1_1, \ldots, a^1_C, \ldots, a^F_1, \ldots, a^F_C), (b^1_1, \ldots, b^1_C, \ldots, b^F_1, \ldots, b^F_C)] \quad (2)$$

where F represents the number of activation functions, and C represents the number of channels of the two input feature vectors $Q^{'}$ and P_n.

Implementation Details: There are two ways to realize the design of a hyper function based on the lightweight network, which is called DS_1 and DS_2 respectively, as shown in Fig. 4a and Fig. 4b. The hyper function can be easily implemented in a similar manner to squeeze-and-excitation [48]. The first way DS_1 for hyper function design is shown in Fig. 4a. In the first step, the two input feature vectors are spliced sequentially. A CNN is used to learn the correlation between the two input feature vectors. The second step is to use global average pooling to compress spatial information and reduce dimensionality. The third step relies on two fully connected layers (with a ReLU between them). ReLU is used to keep the gradient value within a reasonable range in most cases. The fourth step is to normalize the vectors through the normalization layer. Finally, dynamic nonlinear transformation is performed on the two input feature vectors and get the output feature map. The other way DS_2 for hyper function design is shown in Fig. 4b. In the first step, two input feature vectors are spliced by channel. The spatial attention mechanism and CNN are used to learn the correlation between the two input feature vectors. In this method, only the dual-sample correlation calculation way in the first step is different. The rest of the steps share the same steps as the first method.

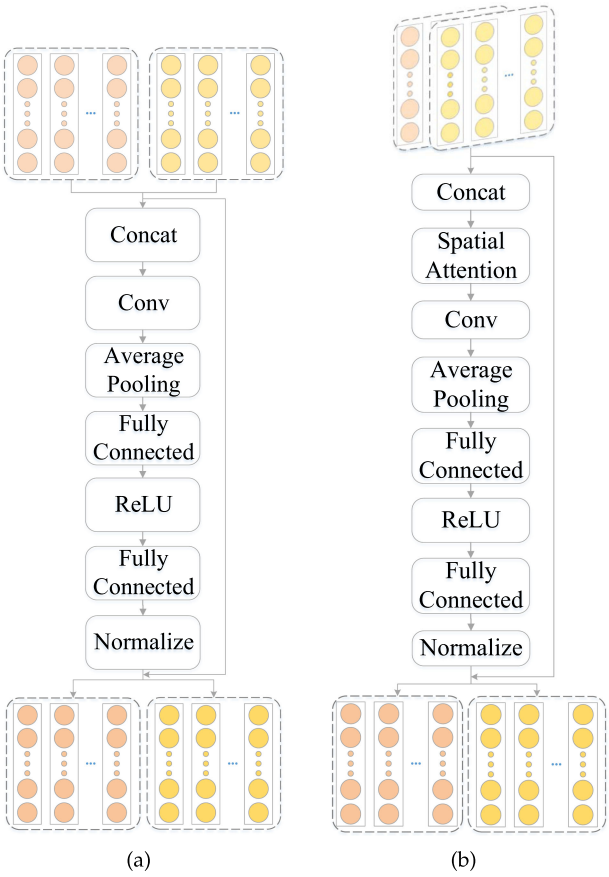


Fig. 4. Implementation details of dual-sample dynamic activation function.

Classification Module

The classification loss is minimized to train a dynamic prototype network based on sample adaptation on the query samples of the training set. In this module, the query samples are classified into N support set classes according to a predefined similarity measure, and the probability that the query sample q_i^w belongs to n^{th} class is predicted as follows:

$$p(y=n|q_i^w) = \frac{exp(-d(\overline{Q}_i^w, \overline{P}_n))}{\sum_{j=1}^{N} exp(-d(\overline{Q}_i^w, \overline{P}_j))} \tag{3}$$

where $d(\cdot,\cdot)$ is the distance function for two given vectors, $Q_i^{w'}$ represent dynamic feature embedding of query samples q_i^w, P_n is the prototype of the n^{th} class. $\overline{Q}_i^w$ and $\overline{P}_n$ are the new feature representations obtained after $Q^{'}$ and P_n input the dual-sample activation function module. In order to be consistent with Fig. 1, we have omitted the superscript and subscript of $Q_i^{w'}$. There are many options for the distance function, such as the cosine distance or the euclidean distance. According to the research by Snell *et al.* [31], the euclidean distance has better performance than other distance functions. Therefore, in this paper, we use the cosine distance to calculate the distance between the query sample and the prototype.

According to the real class label $\widetilde{y} \in \{1, 2, \ldots, N\}$, the negative log probability is used to define the classification loss function in the model training stage, as shown in the following formula. DPNSA can be trained via the classification loss on the query samples is minimized. By optimizing L with the gradient descent algorithm, the network can be trained in an end-to-end manner.

$$L = -\sum_{i=1}^{N}\sum_{w=1}^{W} log\ p(y=\widetilde{y}|q_i^w) \tag{4}$$

In the inductive inference stage, the dynamic embedding module is directly used to extract the dynamic feature embedding of support set samples and query set samples of the unknown new class; then, the class (prototype) features of each unknown are calculated. The class (prototype) features and query sample features are used as the input of the dual-sample dynamic activation module to obtain new features after the interference information is ignored. Finally, the label $\widehat{y}$ of the query sample q_i^w is predicted by discovering the nearest mean class feature under the cosine distance metric

$$\widehat{y} = argmin\ d(\overline{Q}, \overline{P}_n) \tag{5}$$

4 Experiments

In this section, we report the experimental results of few-shot malware detection to prove the effectiveness of our DPNSA model. We also performed ablation experiments to assess the diverse components of our method.

4.1 Experimental Setup

Datasets Description

VirusTotal [55] contains more than 22,270 Windows PE malware files, including more than 500 different classes of malware, such as Trojan horses, viruses, worms, and backdoors. We carry out experiments on a few-shot malware dataset Filtered Large PE Malware, collected by Tang et al [41] from VirusTotal, to assess the effectiveness of our model. There are 100 classes in the meta-training set, 58 classes in the meta-validation set, and 50 classes in the meta-testing set. There are 20 samples for each class. The malware images in this dataset have a size of 256 × 256 pixels.

Experimental Setting

Each epoch has 100 episodes in the meta-training stage, which are randomly sampled from the training set. Each episode consists of N classes and each class contains K support samples. We assessed the effectiveness of our method in 5-way and 10-way settings. In each episode, we set the number of query samples for each type of 5-way 1-shot, 5-way 5-shot, and 5-way 10-shot scenarios for training and inference to 15, and 10, respectively. For the 10-way scenario setting, in each episode, each class uses 5 query samples for training and 4 query samples for inference. This paper adopts the 10-way setting training model to evaluate the 5-way setting, which is the same as [31]. We report the average accuracy (%) and the corresponding 95% confidence interval over the 1,000 episodes randomly sampled from the test set.

Parameters Setting

PyTorch [50] is applied to implement our DPNSA model and compare it with other models. During training, we take in horizontal flip for data augmentation. The Adam algorithm is used as the optimizer. For the DPNSA model, we set the training epochs e at 1000, the initial learning rate lr at 0.001, and the patience p at 200. At the same time, we used the step policy to update the learning rate and set the

TABLE 1
Average Accuracy (%) With 95% Confidence Interval Comparison Between Different Models for Few-Shot Malware Detection on Filtered LargePE Dataset

	Model	5-Way-5 Shot	5-Way 10-Shot
CL	Gist + kNN* [54]	69.07 ± 2.81	75.18 2.53
	N-Gram + kNN*[53]	46.57 ± 2.18	49.37 ± 2.04
	pixel kNN*	63.60 ± 0.94	67.39 ± 0.96
O	SNAIL* [28]	73.81 ± 0.36	75.13 ± 0.35
	Meta-SGD* [27]:	75.54 ± 0.31	77.84 ± 0.31
M	Induction Network* [51]	68.08 ± 0.38	69.17 ± 0.19
	ProtoNet* [31]	79.57 ± 0.22	82.63 ± 0.20
	Relation Network* [32]	74.98 ± 0.23	77.28 ± 0.23
	HABPN* [52]	80.19 ± 0.20	82.24 ± 0.21
	CAN[35]	83.30	80.48
	ConvProtoNet*[41]	83.34 ± 0.13	86.63 ± 0.13
	DPNSA$_{DS_1}$ (5-way)	86.49 ± 0.59 $_{(+3.15)}$	89.30 ± 0.51 $_{(+2.67)}$
	DPNSA$_{DS_1}$ (10-way)	88.60 ± 0.52 $_{(+5.26)}$	90.28 ± 0.47 $_{(+3.65)}$

*The numbers in brackets denote the performance improvement over the best baseline. Result with * are reported in Tang* et al. *[41].*

learning rate linear decay d at 20. This is to say, the learning rate will decay after 20 epochs. All models are trained on the training set, and the best epochs on the validation set are selected to be tested on the test set.

Baselines

To demonstrate the effectiveness, we compare DPNSA with the following baseline methods. The inter-comparable methods are classified into three clusters. Gist+kNN, N-Gram+kNN, and Pixel kNN are classic machine learning-based methods (CL). SNAIL, Meta-SGD are optimization-based methods (O). While Induction Network, Prototypical Network (ProtoNet), Relation Network, Hybrid Attention-Based Protopical Network (HABPN), Cross Attention Network (CAN), and ConvProtoNet are metric learning-based methods (M). A brief similarities and differences of these models has been given in Table 1.

Gist+kNN [54]: Gist+kNN uses Gist descriptors as features and kNN as a classifier.

N-Gram+kNN [53]: N-Gram+kNN extracts byte features using the N-Gram and then uses kNN as the classifier.

Pixel kNN: Pixel kNN is directly used the malware image converted from the binary file as inputs and then used kNN as the classifier.

SNAIL [28]: SNAIL is a class of simple and generic meta-learner architectures that combine interleaved temporal convolution and causal attention.

Meta-SGD [27]: Meta-SGD is a meta-learning method similar to stochastic gradient descent and easy to train. Not only will the learner be initialized for learning, but also the updated direction and learning rate of the learner will be learned. All the processes are completed in a single meta-learning process.

Induction Network [51]: Induction Network establishes a nonlinear mapping from sample to class vectors through a dynamic routing algorithm to learn generalized class representations and measure the correlation between test samples and class representations.

Prototypical Network (ProtoNet) [31]: ProtoNet is a deep metric-based method that calculates each sample vector average as class prototypes.

Relation Network [32]: Relation Network splices the feature vectors of the samples of each class and the test samples and then inputs them into the neural network. CNN is used as the distance metric.

Hybrid Attention-Based Protopical Network (HABPN) [52]: HABPN is a prototype network method based on a hybrid attention mechanism, including sample-level and feature-level attention. It uses a hybrid attention mechanism to alleviate the adverse effects of noisy data and sparse features.

Cross Attention Network (CAN) [35]: A cross-attention map is generated for each couple of class features and query sample features to emphasize the target object region, fabricating the extracted features more discriminative.

ConvProtoNet [41]: ConvProtoNet prevents gradient vanishing problems for few-shot malware detection.

4.2 Experimental Results

We compare our method with other baseline models and report the average accuracy of 5-way 5-shot and 5-way 10-shot settings in Table 1. Our proposed dynamic prototype network method based on sample adaptation has achieved the best results among all methods compared. Compared with the best classic machine learning-based method, our method has an average accuracy increase of 19.53% in the 5-way 5-shot scenario and 15.10% in the 5-way 10-shot scenario. Compared with the best optimization-based method, our method improves the average accuracy by 13.06% in the 5-way 5-shot scenario and by 12.44% in the 5-way 10-shot scenario.

The Gist+KNN model has strong competitiveness based on the classic machine learning method, but there is still a big gap compared with the other two best-performing models. From a macro perspective, we can observe methods based on classical machine learning have the worst performance, the metric-based methods have the best performance, and the performance of methods based on optimization is in between. This is because methods based on classic machine learning need to manually extract features, the quality of which directly affects the result of malware detection. The optimization-based and metric-based methods all train the model in an end-to-end manner, and automatically learn the feature information in the sample without the need for manual feature extraction.

The optimization-based method underperforms the metric-based method because it relies on optimization strategies and adaptability to new samples. The optimization strategies directly affect the result of malware detection. In the optimization-based methods, the target task needs to be fine-tuned, which is time-consuming. It can only be pre-trained and migrated on fixed tasks. For example, models trained on 5-way 1-shot classification tasks can only adapt to 5-way 1-shot tasks, which lacks flexibility. Metric-based methods adopt the feed-forward approach and do not need to be updated that provides a faster, simpler and more effective solution than the above-mentioned methods. Most metric-based methods achieve the best results because this method models the distance between samples so that similar samples are close and heterogeneous samples are kept away. Besides, such methods provide more classes during training than

TABLE 2
Average Accuracy (%) With 95% Confidence Interval in 10-Way for Few-Shot Malare Detection on Filtered LargePE Dataset

Model	10-Way 5-Shot	10-Way 10-Shot
CAN	70.01	69.05
DPNSA_{DS_1} (5-way)	$78.90\pm0.58_{(+8.89)}$	$82.77\pm0.53_{(+13.72)}$
DPNSA_{DS_1} (10-way)	$81.55\pm0.55_{(+2.65)}$	$84.04\pm0.51_{(+1.27)}$

TABLE 3
Average Accuracy(%) With 95% Confidence Interval in 1-Shot for Few-Shot Malware Detection on Filtered LargePE Dataset

Model	5-Way 1-Shot	10-Way 1-Shot
CAN	65.23	56.34
DPNSA_{DS_1} (5-way)	$71.22\pm0.90_{(+5.99)}$	$62.10\pm0.72_{(+5.76)}$
DPNSA_{DS_1} (10-way)	$71.73\pm0.88_{(+0.51)}$	$62.84\pm0.71_{(+0.74)}$

during testing, and providing matching samples during training and testing will lead to better results [31].

Our method is a metric-based method. The existing metric-based methods statically extract the sample features of the support set and the query set separately so that different samples share the same parameters. On the contrary, our DPNSA can realize dynamic feature extraction based on sample adaptation and uses the correlation of the dual-sample to reduce the impact of unrelated features between samples on the metric. We showed one version of DPNSA — DPNSA_{DS_1}, one containing 5 classes (represented by 5-way), and the other containing 10 classes (represented by 10-way). Compared with the optimal metric-based method, on the 5-class training model, our method increased the average test accuracy by 3.15% in the 5-way 5-shot scenario, and by 2.67% in the 5-way 10-shot scenario. On the model trained on 10 classes, our method improved the average accuracy by 5.26% in the 5-way 5-shot scenario, and by 3.65% in the 5-way 10-shot scenario. The empirical evidence shows that both the dynamic feature embedding module and the dual-sample dynamic activation module in our method contribute to the improvement of performance.

Table 2 compares the accuracy achieved by models in the 10-way 5-shot and 10-way 10-shot scenarios. The model that is trained on 10 classes achieves a significantly higher average accuracy than the model trained on 5 classes in the 10-way 5-shot and 10-way 10-shot scenarios.

Table 3 shows the accuracy of models in a 1-shot setting. The model trained in 10 classes achieves an average accuracy of 71.73% in the 5-way 1-shot scenario, and 62.84% in the 10-way 1-shot scenario.

4.3 Ablation Study

In this section, we empirically demonstrate the effectiveness of each component of DPNSA, and the effectiveness of different implementations of the dual-sample dynamic activation function. All experiments are performed on the Filtered LargePE dataset.

Influence of Different Dual-Sample Dynamic Activation Function Implementation Ways

As previously mentioned, we implemented two different DPSNA variants based on the DS_1 and DS_2 proposed in Section 3 respectively, namely DPNSA_{DS_1} and DPNSA_{DS_2}. We learn to express the correlation feature between two samples for DPNSA_{DS_2} by fusing the corresponding position information between the samples. Quite the opposite, we do not intervene in the two-sample correlation feature fusion method for DPNSA_{DS_1}, but adopt an end-to-end approach. The comparison result is shown in Table 4 that DPNSA_{DS_1} outperforms DPNSA_{DS_2} regardless of the few-shot scenario setting. This shows the end-to-end approach is more conducive to the extraction of key correlation feature representations between two samples.

Influence of Dynamic Convolution and Dual-Sample Dynamic Activation Function

We prove the effectiveness of the dynamic embedding module and the dual-sample dynamic activation module here. As shown in Table 5, Basic Network represents the prototype network implemented in this paper without including dynamic convolution and dual-sample dynamic activation function; $\text{DPNSA}_{(ND)}$ indicates the DPNSA model that considers only the dual-sample dynamic activation function and dispenses with the dynamic convolutional neural network. $\text{DPNSA}_{(ND)}$ outperforms the Basic Network based on the

TABLE 4
Ablation Study of Different Implementation Ways of the Dual-Sample Dynamic Activation Function on Filtered LargePE Dataset

Model	5-Way 1-Shot	5-Way 5-Shot	5-Way 10-Shot	10-way 1-shot	10-way 5-shot	10-way 10-shot
DPNSA_{DS_2} (5-way)	69.44 ± 0.93	85.82 ± 0.60	87.39 ± 0.54	61.02 ± 0.73	78.16 ± 0.59	80.01 ± 0.55
DPNSA_{DS_2} (10-way)	70.62 ± 0.90	86.97 ± 0.58	89.20 ± 0.51	61.70 ± 0.73	79.68 ± 0.58	82.86 ± 0.53
DPNSA_{DS_1} (5-way)	$71.22\pm0.90_{(+1.78)}$	$86.49\pm0.59_{(+0.67)}$	$89.30\pm0.51_{(+1.91)}$	$62.10\pm0.72_{(+1.08)}$	$78.90\pm0.58_{(+0.74)}$	$82.77\pm0.53_{(+2.76)}$
DPNSA_{DS_1} (10-way)	$71.73\pm0.88_{(+1.11)}$	$88.60\pm0.52_{(+1.63)}$	$90.28\pm0.47_{(+1.08)}$	$62.84\pm0.71_{(+1.14)}$	$81.55\pm0.55_{(+1.87)}$	$84.04\pm0.51_{(+1.18)}$

TABLE 5
Ablation Study on the Effectiveness of Different Components of the DPNSA Model on Filtered LargePE Dataset

Model	5-Way 1-Shot	5-Way 5-Shot	5-Way 10-Shot	10-way 1-shot	10-way 5-shot	10-way 10-shot
Basic Network	64.76 ± 0.94	83.97 ± 0.64	85.85 ± 0.57	56.14 ± 0.66	77.50 ± 0.59	80.66 ± 0.54
$\text{DPNSA}_{(ND)}$ (5-way)	$70.72\pm0.90_{(+5.96)}$	$86.49\pm0.58_{(+2.52)}$	$87.94\pm0.54_{(+2.09)}$	$61.79\pm0.71_{(+5.65)}$	$79.20\pm0.59_{(+1.70)}$	$80.72\pm0.57_{(+0.06)}$
$\text{DPNSA}_{(ND)}$ (10-way)	$73.02\pm0.89_{(+2.30)}$	$87.95\pm0.54_{(+1.46)}$	$89.93\pm0.49_{(+1.99)}$	$63.82\pm0.71_{(+2.03)}$	$81.00\pm0.57_{(+1.80)}$	$83.44\pm0.52_{(+2.72)}$
DPNSA (5-way)	$71.22\pm0.90_{(+0.50)}$	$86.49\pm0.59_{(+0.00)}$	$89.30\pm0.51_{(+1.36)}$	$62.10\pm0.72_{(+0.31)}$	$78.90\pm0.58_{(-0.30)}$	$82.77\pm0.53_{(+2.05)}$
DPNSA (10-way)	$71.73\pm0.88_{(-1.29)}$	$88.60\pm0.52_{(+0.65)}$	$90.28\pm0.47_{(+0.35)}$	$62.84\pm0.71_{(-0.98)}$	$81.55\pm0.55_{(+0.55)}$	$84.04\pm0.51_{(+0.60)}$

static activation function in a variety of few-shot scenarios. This is because the traditional static activation function performs the same nonlinear transformation for different input samples. In contrast, according to the correlation between the samples, the dual-sample dynamic activation function adaptively learns a new activation function for the two samples. Each sample is dynamically activated to achieve dynamic nonlinear transformation by the new activation function learned. It can effectively decrease the influence of irrelevant features, and thereby increase the metric accuracy. When dynamic convolution and the dual-sample dynamic activation function are considered simultaneously, the dynamic feature embedding has better performance than static feature embedding. These both have consistent improvements in many few-shot scenario settings. This is because dynamic convolution can realize dynamic feature embedding of samples by adaptively adjusting convolution parameters through input samples.

The performance gap also proves the following points: (1) In a few-shot scenario, due to the scarcity of available samples, it is necessary to fully utilize and mine the semantic information of the samples, and adjust the convolution parameters adaptively according to the input samples for dynamic feature embedding; (2) the dual-sample dynamic activation function can help ignore the interference information between samples, reduce the impact of irrelevant features on the measurement, and improve the distinguishability of the learned features, thereby improving the metric accuracy. Experimental results demonstrate that the dynamic feature embedding module and the dual-sample dynamic activation function module are both effective for increasing the performance of the model.

5 Conclusion

In this paper, we propose a dynamic prototype network based on sample adaptation for few-shot malware detection. First, the dynamic convolutional neural network is introduced to realize dynamic feature embedding based on sample adaptation to extract richer semantic information from each sample. Then, we proposed the dual-sample dynamic activation function, which uses the correlation of two samples to dynamically activate the input malware class features (prototype) and query sample features. It can effectively ignore the interference information between samples and decrease the impact of irrelevant features on the metric. Finally, a metric-based method is used to calculate the distance between the query sample and the malware class prototype to realize effective malware detection. A great many of experiments have demonstrated that our model is more effective than the baseline few-shot malware detection models and has achieved excellent results.

In the future, we will further probe into the following directions:

1) When the new task domain differs from the old one, the existing few-shot learning method shows limited transferability. It cannot achieve good results in new tasks domains. Due to the shortage of data for network security and the high cost of data labelling, we will explore to improve the model's generalization capacity and enable models trained on data from other fields to work out the issue of few-shot malware detection.
2) In real-world applications, a malware detection system needs to be capable of detecting not only known malware, but also unknown malware in real-time. Therefore, we will explore the implementation of incremental few-shot malware detection methods to meet the requirements in real-life scenarios.

References

[1] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. USENIX Secur. Symp.*, 2019, pp. 729–746.

[2] X. H. Zhang *et al.*, "Enhancing state-of-the-art classifiers with API semantics to detect evolved android malware," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 757–770.

[3] D. Gibert, C. Mateu, and J. Planes, "A hierarchical convolutional neural network for malware classification," in *Proc. Int. Joint Conf. Neural Netw.*, 2019, pp. 1–8.

[4] J. Yan, G. Yan, and D. Jin, "Classifying malware represented as control flow graphs using deep graph convolutional neural network," in *Proc. 49th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2019, pp. 52–63.

[5] S. Tobiyama, Y. Yamaguchi, H. Shimada, T. Ikuse, and T. Yagi, "Malware detection with deep neural network using process behavior," in *Proc. IEEE 40th Annu. Comput. Softw. Appl. Conf.*, 2016, pp. 577–582.

[6] X. F. Lu, F. S. Jiang, X. Zhou, S. W. Yi, J. Sha, and L. Pietro, "ASSCA: API sequence and statistics features combined architecture for malware detection," *Comput. Netw.*, vol. 157, pp. 99–111, 2019.

[7] F. Xiao, Z. W. Lin, Y. Sun, and Y. Ma, "Malware detection based on deep learning of behavior graphs," *Math. Problems Eng.*, vol. 2019, 2019, Art. no. 195395.

[8] J. Y. Kim, S. J. Bu, and S. B. Cho, "Zero-day malware detection using transferred generative adversarial networks based on deep autoencoders," *Inf. Sci.*, vol. 460, pp. 83–102, 2018.

[9] N. Bhodia, P. Prajapati, F. Di Troia, and M. Stamp, "Transfer learning for image-based malware classification," 2019, *arXiv: 1903.11551*.

[10] L. Yao, C. Mao, and Y. Luo, "Graph convolutional networks for text classification," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 7370–7377.

[11] L. Z. Huang, D. H. Ma, S. J. Li, X. D. Zhang, and H. F. Wang, "Text level graph neural network for text classification," in *Proc. Conf. EMNLP-IJCNLP*, 2019, pp. 3444–3450.

[12] Y. F. Zhang, X. L. Yu, Z. Y. Cui, S. Wu, Z. Z. Wen, and L Wang, "Every document owns its structure: Inductive text classification via graph neural networks," in *Proc. Conf. ACL*, 2020, pp. 334–339.

[13] Z. Yang, T. G. Luo, D. Wang, Z. Q. Hu, J. Gao, and L. W. Wang, "Learning to navigate for fine-grained classification," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 420–435

[14] Y. Gao, X. T. Han, X. Wang, W. L. Huang, and M. R. Scott, "Channel interaction networks for fine-grained image categorization," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 10 818–10 825.

[15] R. Pascanu, J. W. Stokes, H. Sanossian, M. Marinescu, and A. Thomas, "Malware classification with recurrent networks," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2015, pp. 1916–1920.

[16] B. Kolosnjaji, A. Zarras, G. Webster, and C. Eckert, "Deep learning for classification of malware system call sequences," in *Proc. Australas. Joint Conf. Artif. Intell.*, 2016, pp. 137–149.

[17] Y. H. Chai, J. Qiu, S. Su, C. S. Zhu, L. H. Yin, and Z. H. Tian, "LGMal: A joint framework based on local and global features for malware detection," in *Proc. IEEE Int. Wireless Commun. Mobile Comput.*, 2020, pp. 463–468.

[18] Z. Q. Zhang, P. P. Qi, and W. Wang, "Dynamic malware analysis with feature engineering and feature learning," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 1210–1217.

[19] D. Gibert, C. Mateu, and J. Planes, "HYDRA: A multimodal deep learning framework for malware classification," *Comput. Secur.*, vol. 95, 2020, Art. no. 101873.

[20] Y. S. Yen, Z. W. Chen, Y. R. Guo, and M. C. Chen, "Integration of static and dynamic analysis for malware family classification with composite neural network," 2019, *arXiv:1912.11249*.

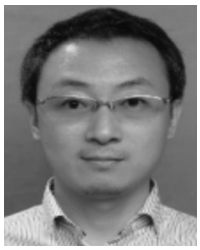
[21] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, "Malware images: Visualization and automatic classification," in *Proc. 8th Int. Symp. Vis. Cyber Secur.*, 2011, pp. 1–7.
[22] B. G. Yuan, J. F. Wang, D. Liu, W. Guo, P. Wu, and X. H. Bao, "Byte-level malware classification based on Markov images and deep learning," *Comput. Secur.*, vol. 92, 2020, Art. no. 101740.
[23] E. Snow, M. Alam, A. Glandon, and K. Iftekharuddin, "End-to-end multimodel deep learning for malware classification," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, 2020, pp. 1–7.
[24] K. He and D. S. Kim, "Malware detection with malware images using deep learning techniques," in *Proc. IEEE Int. Conf. Trust Secur. Privacy Comput. Commun./13th IEEE Int. Conf. Big Data Sci. Eng.*, 2019, pp. 95–102.
[25] T. Hospedales, A. Antoniou, P. Micaelli, and A. Storkey, "Meta-learning in neural networks: A Survey," 2020, *arXiv:2004.05439*.
[26] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1126–1135.
[27] Z. G. Li, F. W. Zhou, F. Chen, and H. Li, "Meta-SGD: Learning to learn quickly for few-shot learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1–11.
[28] N. Mishra, M. Rohaninejad, X. Chen, and P. Abbeel, "A simple neural attentive meta-learner," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–17.
[29] M. A. Jamal and G. J. Qi, "Task agnostic meta-learning for few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11719–11727.
[30] A. Nichol, J. Achiam, and J. Schulman, "On first-order meta-learning algorithms," 2018, *arXiv:1803.02999*.
[31] J. Snell, K. Swersky, and R. S. Zemel, "Prototypical networks for few-shot learning," in *Proc. Conf. Neural Inf. Process. Syst. Workshop*, 2017, pp. 4080–4090.
[32] F. Sung, Y. X. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning,"in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1199–1208.
[33] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra, "Matching networks for one shot learning," in *Proc. Conf. Neural Inf. Process. Syst. Workshop*, 2016, pp. 3630–3638.
[34] B. N. Oreshkin, P. Rodriguez, and A. Lacoste, "Tadam: Task dependent adaptive metric for improved few-shot learning," in *Proc. Conf. Neural Inf. Process. Syst. Workshop*, 2018, pp. 1–13.
[35] R. B. Hou, H. Chang, B. P. Ma, S. G. Shan, and X. L. Chen, "Cross attention network for few-shot classification," in *Proc. Conf. Neural Inf. Process. Syst. Workshop*, 2019, pp. 4003–4014.
[36] H. Y. Tseng, H. Y. Lee, J. B. Huang, and M. H. Yang, "Cross-domain few-shot classification via learned feature-wise transformation," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–18.
[37] H. J. Ye, H. X. Hu, D. C. Zhan, and F. Sha, "Few-shot learning via embedding adaptation with set-to-set function," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 8808–8817.
[38] Y. Wang *et al.*, "Large margin meta-learning for few-shot classification," in *Proc. 2nd Workshop Meta-Learn Conf. Neural Inf. Process. Syst.*, 2018, pp. 1–8.
[39] S. Gidaris, A. Bursuc, N. Komodakis, P. Pérez, and M. Cord, "Boosting few-shot visual learning with self-supervision," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 8059–8068.
[40] C. Zhang, Y. J. Cai, G. S. Lin, and C. H. Shen, "DeepEMD: Few-shot image classification with differentiable Earth mover's distance and structured classifiers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 12 203–12 213.
[41] Z. J. Tang, P. Wang, and J. F. Wang, "ConvProtoNet: Deep prototype induction towards better class representation for few-shot malware classification," *Appl. Sci.*, vol. 10, no. 8, 2020, Art. no. 2847.
[42] Y. D. Bai, Z. C. Xing, X. H. Li, Z. Y. Feng, and D. Y. Ma, "Unsuccessful story about few shot malware family classification and siamese network to the rescue," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng.*, 2020, pp. 1560–1571.
[43] K. Tran, H. S. Sato, and M. Kubo, "MANNWARE: A malware classification approach with a few samples using a memory augmented neural network," *Information*, vol. 11, no. 1, 2020, Art. no. 51.
[44] L. T. Deng, H. Wen, M. F. Xin, Y. Sun, L. M. Sun, and H. S. Zhu, "Malware classification using attention-based transductive learning network," in *Proc. Int. Conf. Secur. Privacy Commun. Syst.*, 2020, pp. 403–418.
[45] J. T. Zhu, J. L. Jang-Jaccard, and P. A. Watters, "Multi-loss siamese neural network with batch normalization layer for malware detection," *IEEE Access*, vol. 8, pp. 171 542–171 550, 2020.
[46] S. C. Hsiao, D. Y. Kao, Z. Y. Liu, and R. L. Tso, "Malware image classification using one-shot learning with siamese networks," *Procedia Comput. Sci.*, vol. 159, pp. 1863–1871, 2019.
[47] Y. P. Chen, X. Y. Dai, M. C. Liu, D. D. Chen, L. Yuan, and Z. C. Liu, "Dynamic convolution: Attention over convolution kernels,"in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11 030–11 039.
[48] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7132–7141.
[49] Y. P. Chen, X. Y. Dai, M. C. Liu, D. D. Chen, L. Yuan, and Z. C. Liu, "Dynamic ReLU," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 351–367.
[50] A. Paszke *et al.* "Automatic differentiation in PyTorch," in *Proc. Conf. Neural Inf. Process. Syst. Workshop*, 2017, pp. 1–4.
[51] R. Y. Geng, B. H. Li, Y. B. Li, X. D. Zhu, P. Jian, and J. Sun, "Induction networks for few-shot text classification," in *Proc. Conf. EMNLP-IJCNLP*, 2019, pp. 3895–3904.
[52] T. Y. Gao, X. Han, Z. Y. Liu, and M. S. Sun, "Hybrid attention-based prototypical networks for noisy few-shot relation classification," *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 01, pp. 6407–6414, 2019.
[53] T. Abou-Assaleh, N. Cercone, V. Keselj, and R. Sweidan, "N-gram-based detection of new malicious code," in *Proc. 28th Annu. Int. Comput. Softw. Appl. Conf.*, 2004, pp. 41–42.
[54] S. Yajamanam, V. R. S. Selvin, F. Di Troia, and M. Stamp, "Deep learning versus gist descriptors for image-based malware classification," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 553–561.
[55] H. Sistemas, Virustotal-free online virus and malware scan, 2004. [Online]. Available: https://www.virustotal.com/en
[56] H. J. Zhu, L. M. Wang, S. Zhong, Y. Li, and V. S. Sheng, "A hybrid deep network framework for android malware detection," *IEEE Trans. Knowl. Data Eng.*, early access, Mar. 26, 2021, doi: 10.1109/TKDE.2021.3067658.

Yuhan Chai received the master's degree from the Hebei University of Science and Technology, Shijiazhuang, China, in 2019. Her current research interests include deep learning and malware detection. For more information, please visit chaiyuhan@e.gzhu.edu.cn.

Lei Du received the master's degree from the Hebei University of Science and Technology, Shijiazhuang, China, in 2021. His current research interests include deep learning and network security. For more information, please visit leidu_close@163.com.

Jing Qiu received the PhD degree in computer applications technology from Beijing Institute of Technology, Beijing, China. She is currently a professor with the Cyberspace Institute of Advanced Technology, Guangzhou University. She was a visiting scholar with the University of Southern California, LA, under the supervision of Professor Craig A. Knoblock. Her current research interests include Information Extraction, Network Representation, and Big Data Analysis. For more information, please visit qiujing@gzhu.edu.cn.

Lihua Yin received the PhD degree from the Harbin Institute of Technology, Harbin, China. She is currently a professor, and associate dean, with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangdong Province, China. She has authored more than 100 papers published by referred international conferences and journals. Her research interests include computer networks, Internet of Things, and cyberspace security. Her research works has been supported by the National Natural Science Foundation of China, National Key research and Development Plan of China, the Major State Basic Research Development Program of China (973 Program), and National High-tech R&D Program of China (863 Program). She also served as a member of China Computer Federation. For more information, please visit yinlh@gzhu.edu.cn.



Zhihong Tian (Senior Member, IEEE) received the BS, MS, and PhD degrees in computer science and technology from the Harbin Institute of Technology, Harbin, China in 2001, 2003, and 2006 respectively. He is currently a professor, and dean, with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangdong Province, China. He is honored as Pearl River Scholar in Guangdong Province. He is also a part-time professor at Carlton University, Ottawa, Canada. Previously, he served in different academic and administrative positions at the Harbin Institute of Technology. He has authored more than 200 journal and conference papers. His research interests include computer networks and cyberspace security. His research works has been supported by the National Natural Science Foundation of China, National Key research and Development Plan of China, and National High-tech R&D Program of China (863 Program). He also served as a member, chair, and general chair of a number of international conferences. He is a distinguished member of the China Computer Federation. For more information, please visit tianzhihong@gzhu.edu.cn.